

Table 1: Netfilter tables and their corresponding hooks

	HOOKS
Filter	NF_INET_LOCAL_IN, NF_INET_FORWARD, NF_INET_LOCAL_OUT
Nat	NF_INET_PRE_ROUTING, NF_INET_POST_ROUTING, NF_INET_LOCAL_OUT
mangle	NF_INET_PRE_ROUTING, NF_INET_LOCAL_IN, NF_INET_FORWARD, NF_INET_POST_ROUTING, NF_INET_LOCAL_OUT

We now have our firewall implemented as a kernel module in `icmp_drop.c`. Let's compile with the following Makefile:

```
1 obj-m += icmp_drop.o
2 all:
3     make -C /lib/modules/$(shell uname -r)/build
M=$(PWD) modules
4 clean:
5     make -C /lib/modules/$(shell uname -r)/build
M=$(PWD) clean
```

Issuing `make` builds our kernel module, i.e., `icmp_drop.ko`. On `insmod` the `icmp_drop.ko`, our firewall is active and blocks all the ICMP ping packets from the machine 192.168.164.51 as shown in `dmesg` output.



Note: Issue `make` from the directory containing the `Makefile` for our source `icmp_drop.c`

dmesg output

```
[root@roy src]# dmesg -c
[1575576.005547] Loading icmp_drop.ko
[1575574.548603] NF_INET_PRE_ROUTING : Dropping the ICMP packet# 1
[1575575.539673] NF_INET_PRE_ROUTING : Dropping the ICMP packet# 2
[1575576.539618] NF_INET_PRE_ROUTING : Dropping the ICMP packet# 3
[1575577.539538] NF_INET_PRE_ROUTING : Dropping the ICMP packet# 4
[1575578.540087] NF_INET_PRE_ROUTING : Dropping the ICMP packet# 5
[1575579.539407] NF_INET_PRE_ROUTING : Dropping the ICMP packet# 6
[1575580.539328] NF_INET_PRE_ROUTING : Dropping the ICMP packet# 7
[1575581.539234] NF_INET_PRE_ROUTING : Dropping the ICMP packet# 8
[1575582.539152] NF_INET_PRE_ROUTING : Dropping the ICMP packet# 9
[1575583.539076] NF_INET_PRE_ROUTING : Dropping the ICMP packet# 10
[root@roy src]# rmmod icmp_drop
[root@roy src]# dmesg
[1575587.001203] Unloading icmp_drop.ko
[1575587.001211] Total ICMP packets dropped: 10
```

Observe that in the above capture, no ICMP response is sent from the machine 192.168.164.39 as ICMP request packets are being dropped by our firewall.

3) **Iptables and tables in Netfilter:** Every systems administrator will be familiar with the Linux firewall solution - Iptables. Netfilter is the engine behind this famous and flexible firewall tool, and is built on top of the Netfilter framework. Iptables comprises a kernel component—*iptables*, and a user land configuration tool—*iptables*. Iptables rely on the 'tables' defined by Netfilter integrated at different

hooks of the framework. Currently, Netfilter defines three default tables—the filter, nat and mangle tables. Table 1 lists the Netfilter tables and corresponding hooks.

filter table – The table is hooked at `LOCAL_IN`, `FORWARD` and `LOCAL_OUT` hooks. As the name suggests, the table filters the packets matched by the rules inserted into the table through the `iptables` command from the shell. Among the three tables, the filter table has the highest priority; hence, it is the last of the tables to get the packets. Note that the userland `iptables` command refers to hooks as chains, viz – `PRE_ROUTING` chain, `OUTPUT CHAIN`, `FORWARD` chain and so on.

nat table - This is hooked at `PRE_ROUTING`, `POST_ROUTING` and `LOCAL_OUT`. The `nat` table is used to configure `SNAT` and `DNAT` and masquerades through the `iptables` command. The `nat` table has a lower priority than the filter tables and hence receives the packets prior to the filter table.

mangle table – This is hooked with all the five hooks (chains, in the iptables context) and is used to modify the `TOS`, `TTL` or mark the packet (`FWMARK`). The mangle table has the lowest priority and hence is the first to receive the packets.

Also, the Netfilter/iptables framework provides us extension facilities, using which kernel modules can register new tables and ask the packets to traverse them.

Summing up the Netfilter journey

In the journey so far, we acquired a basic understanding of a powerful packet hacking framework called Netfilter. We even got our hands dirty, rolled out our own firewall and touched on a few points on iptables. Apparently, Netfilter/iptables can do a lot more to satisfy our potential needs. The power and versatility of Netfilter is limited only by our imagination and creativity.

Happy hacking! 

Acknowledgement:

I would like to thank my mentors Subhash KJ and Vinay Sagar of MindSculptor Systems for their guidance.

References

- [1] <http://www.netfilter.org/>
- [2] <http://www.linuxjournal.com/article/7184?page=0,0>

Roysten Serrao

The author works as a software engineer in a product development company. He is a Linux kernel and systems software enthusiast.